Stochastic Models and Simulation

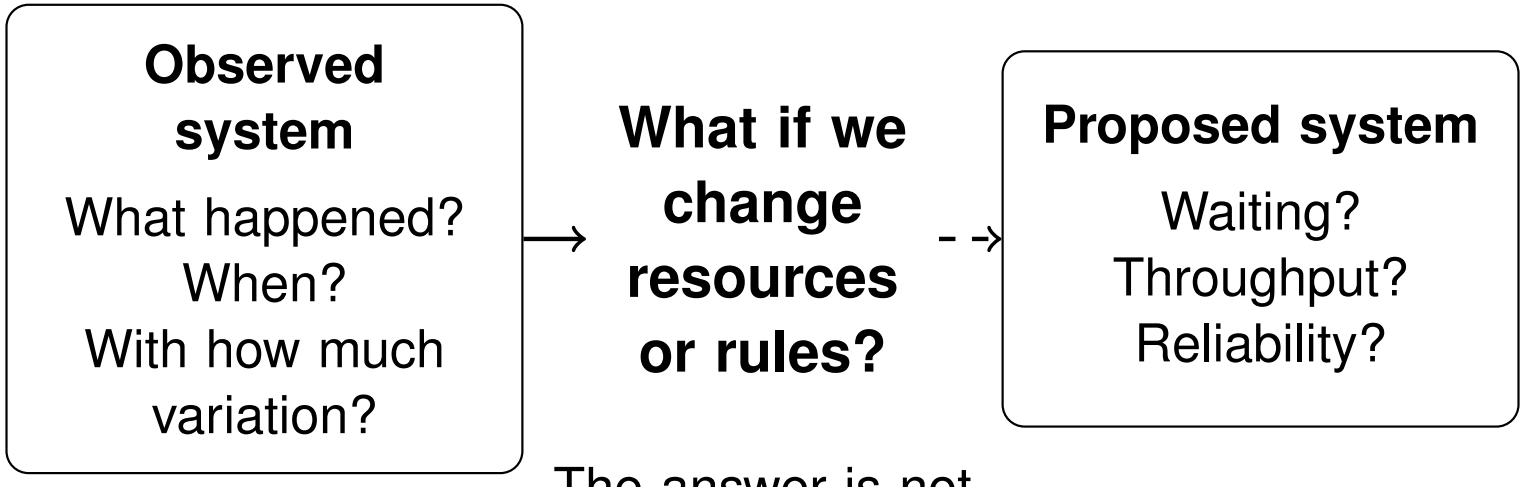
Week 1: from observed calls to a testable model

Sebastian Müller

2026/27



Why this course? What do you gain?



The answer is not in the records.

You learn to turn this gap into a defensible decision.

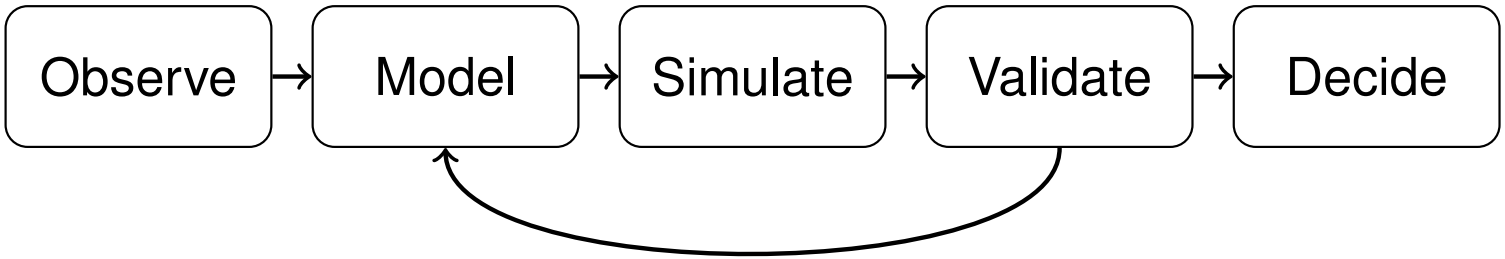
Define the outcome, expose the assumptions, quantify uncertainty, and explain what the evidence can support.

A question the existing records cannot answer

Would another agent reduce waiting at the bank's call centre?

- The records describe what happened with the staffing and routing that were in place.
- They contain no observation of the proposed staffing change.
- A simulation can explore that change, *if* its assumptions survive comparison with observed behaviour.

The course in one loop



- **This week:** test an arrival-count model against a real call-centre extract.
- **Across seven sessions:** add queues, estimation, uncertainty, model choice and a project decision.

Rules of the game: every three-hour session

1. **Lecture segment:** screens closed, phones off, no side conversations.
2. **Notebook and exercises:** open your own computer; use Python and LLMs, inspect data and record your reasoning. Discuss with your group, but do not copy code or answers.
3. **Final 10 minutes:** complete a printed multiple-choice quiz individually, without devices or LLMs.

Working with an LLM: use it or lose it

Use it deliberately

- Delegate a precise, well-defined task.
- Ask follow-up questions and request alternatives.
- Use the exchange to dig deeper and increase your understanding.

Stay sceptical

- A one-shot answer may be wrong or merely acceptable.
- Check claims with mathematics, Python or an independent source.
- You own the final reasoning and must be able to explain it.

Exit Quizz

In the final 10 minutes, complete the printed exit quiz handed out in class.

- Work individually, without devices or LLMs.
- Choose one answer per question.
- Give one question about the data or model that you still have.

The quiz checks basic understanding of the lecture and data used, basic Python commands, variability and independent LLM verification.

Assessment

- Graded end-of-session MCQs are scored out of 20. Their number and schedule are to be confirmed. An absence scores zero.
- Let N be the number of scheduled quizzes, including any missed. Drop the *single lowest* score (one free card):

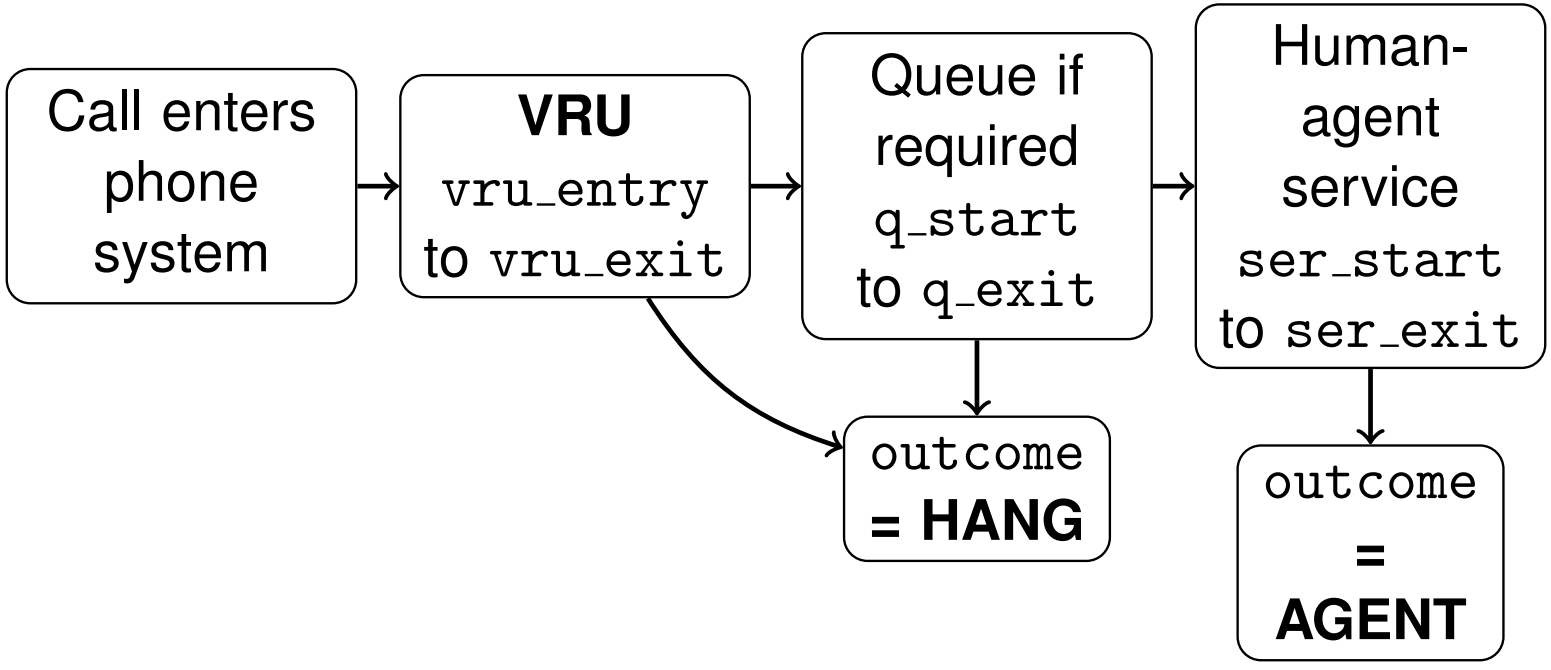
$$\mathrm{CC} = \frac{\text{sum of scores} - \text{lowest score}}{N - 1}$$

- Final exam E is scored out of 20. The final mark is the the average of E and CC:

$$\mathrm{Final} = \frac{E + \mathrm{CC}}{2}$$

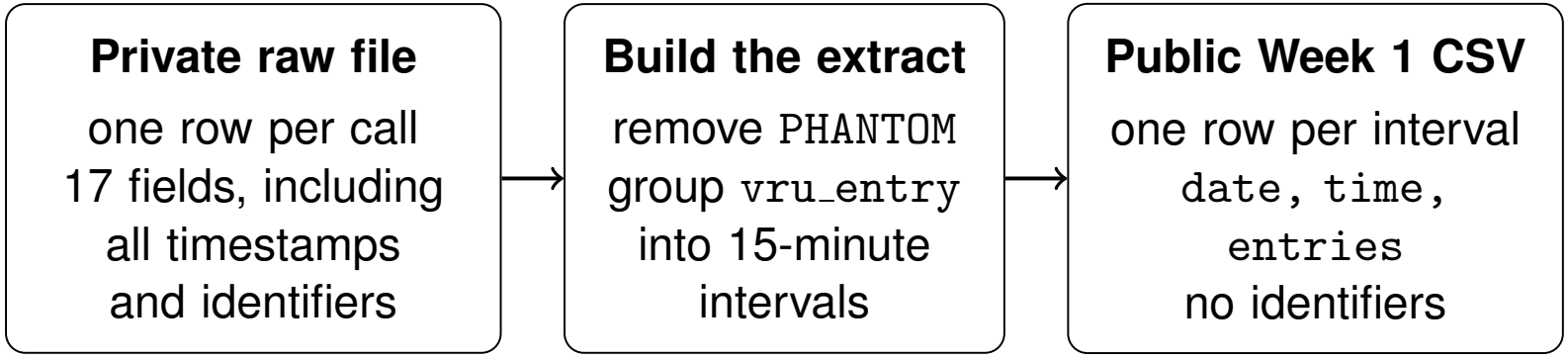
Example if $N = 6$: $0, 10, 12, 14, 16, 18 \to$ drop 0, so $\mathrm{CC} = 14$. If exam mark $E = 12$, the final mark is $\mathit{mean}(12, 14) = 13$.

How one call is recorded



- The top row contains **stages and their timestamps**; a call can skip the queue.
- The lower boxes contain one categorical result: `AGENT` if service occurred, or `HANG` if the caller left first.
- `AGENT` is not the employee. The employee name is stored in `server`; `PHANTOM` records are discarded.

From the raw call log to the Week 1 CSV



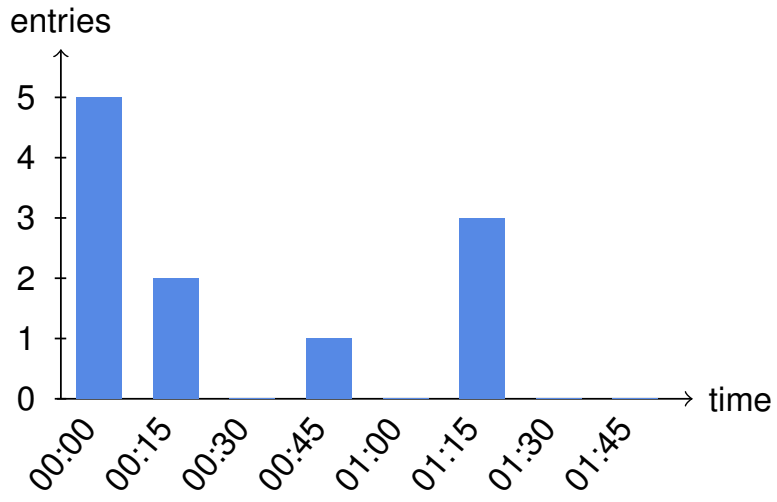
- `entries` counts retained calls by `vru_entry`; `1999-11-01,10:00,49` means 49 calls entered the VRU in [10:00, 10:15).
- The other timestamps—`vru_exit`, queue start/exit and service start/exit—remain only in the private raw file. They are **not** columns of this CSV.
- The aggregate has 2,880 intervals and 40,722 entries. A VRU entry is not necessarily a queue arrival or completed service.

Dataset guide: all raw fields, redacted examples, provenance and limitations.

The first rows of the Week 1 CSV

date	time	entries
1999-11-01	00:00	5
1999-11-01	00:15	2
1999-11-01	00:30	0
1999-11-01	00:45	1
1999-11-01	01:00	0
1999-11-01	01:15	3
1999-11-01	01:30	0
1999-11-01	01:45	0

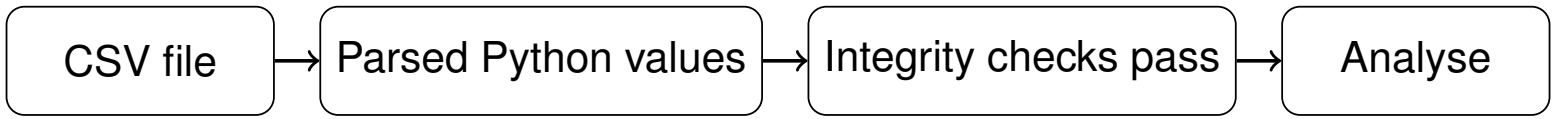
One row represents one 15-minute interval. Zero is an observed count, not a missing value.



Eight consecutive entry counts from the start of 1 November.

Interpretation: five retained calls entered the VRU from 00:00 to 00:14:59; this is not the number waiting in a queue.

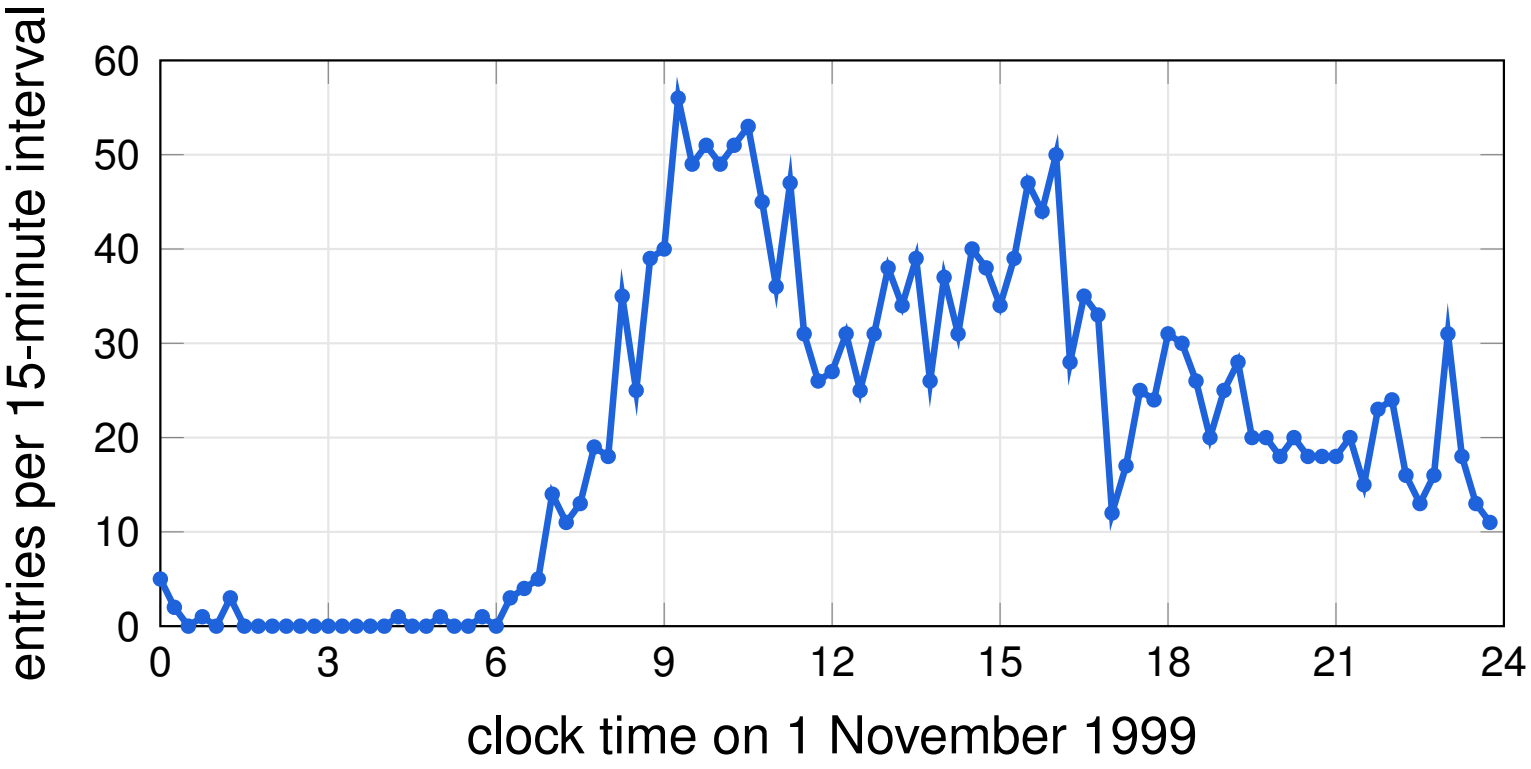
Before modelling: verify the input



- Convert `entries` from CSV text to integers.
- Verify one row for every expected interval: 30 days $\times$24 hours $\times$4 quarter-hours $=$ 2,880, with no duplicated or missing date-time pair. Otherwise rates and time profiles would be distorted.
- Verify that every count is nonnegative and that the total is 40,722, as documented by the build process. Otherwise the file or parsing step is wrong.

Why? If a check fails, a later discrepancy may come from the file or our code. These checks validate the input pipeline.

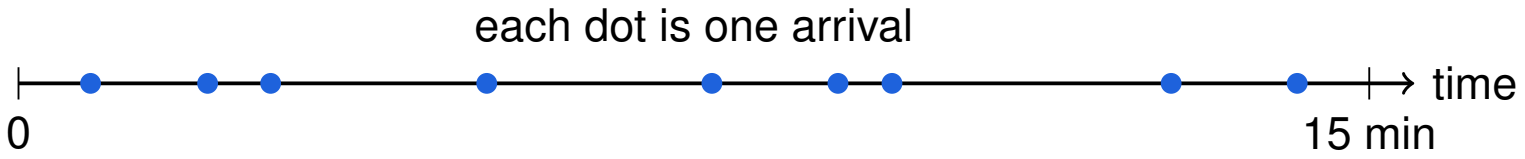
Observed 15-minute counts on 1 November



The 96 points are the 96 CSV rows for this date: no moving average and no averaging across days.

First pattern: the arrival rate changes during the day. The full month can then reveal a second pattern: whether the same time of day behaves differently on different days or weekdays.

Poisson process: the basic idea



Homogeneous-process assumptions

- Arrivals occur at a constant average rate r.
- Counts in disjoint intervals are independent.
- In a very short interval, two or more arrivals are negligible.

Consequences for an interval of length t

$$
\begin{aligned}
N(t) &\sim \text{Poisson}(rt), \\
\mathbb{E}[N(t)] &= \text{Var}(N(t)) = rt, \\
T_{\text{next}} &\sim \text{Exponential}(r), \\
\mathbb{E}[T_{\text{next}}] &= 1/r.
\end{aligned}
$$

A Poisson **process** models arrival times. The Poisson **distribution** describes how many arrivals fall inside a chosen interval.

First baseline: one rate for every interval

Deliberately ignore clock time and date. Assume the same constant rate r for all 2,880 intervals in November. For each 15-minute count X_i:

$$X_i \sim \text{Poisson}(15r), \qquad \mathbb{E}[X_i] = \text{Var}(X_i) = 15r,$$
$$\mathbb{P}(X_i = 0) = e^{-15r}.$$

- The homogeneous Poisson process also assumes independent counts in disjoint intervals.
- Fit r from the mean of all 2,880 counts: $\hat{r} = 14.14/15 \approx 0.943$ entries per minute.
- This deliberately pools nights, busy hours, weekdays and weekends. We use it as a simple baseline whose failures should reveal which variation matters.

The pooled baseline fails two checks

	Observed	**Constant-rate model**
Mean count	14.14	14.14 (fitted)
Variance of counts	209.30	14.14
Empty-bin fraction	29.1%	$e^{-14.14} \approx 7.2 \times 10^{-7}$

- The observed variance uses divisor n (Python `statistics.pvariance`).
- The pooled model predicts almost no empty bins; the extract has many.
- This challenges a *constant* rate. We must inspect when the empty bins occur.

Simulate arrivals with plain Python

```
import random, statistics
rate = statistics.mean(observed) / 15  # per minute
rng = random.Random(2026)
simulated = [0] * len(observed)
clock = rng.expovariate(rate)
while clock < 15 * len(observed):
    simulated[int(clock // 15)] += 1
    clock += rng.expovariate(rate)
```

Each exponential gap moves the clock to the next arrival. The notebook wraps this in a reusable function with input checks.

Check the simulator; then check the model

One run of the preceding code: 2,880 intervals, seed 2026.

	Model prediction	**Simulated**	**Observed**
Mean count	14.14	14.25	14.14
Variance of counts	14.14	14.52	209.30
Empty intervals	0.000072%	0%	29.1%

- **Simulator check:** the simulated mean and variance are close to the theoretical values. This supports the implementation; repeat with other seeds.
- **Model check:** the observed counts have much greater variance and many more empty intervals. The constant-rate baseline misses this variation.

Results vary between runs. Choose the seed before comparing results; do not tune it to match the data. Variances above use divisor n.

Validate by clock hour, then propose a next test

- The homogeneous model expects about 14.14 entries per 15-minute bin at *every* clock hour.
- Observed means: about 0.1 at 03:00 and 32.3 at 10:00, across all November days.
- Find where the notebook's simulated hourly profile overpredicts and underpredicts the data.

Two sources of variation: first estimate a rate for each time of day; then check whether those rates also differ between weekdays, weekends or individual dates. Reserve later days to test the fitted rates. A better daily profile alone would not prove independent Poisson arrivals.

What can we decide today?

Today's goal:

- Describe when VRU entries occur.
- Test a simple model for entry counts.
- Decide which assumption needs a better check.

Next steps:

- Agent-directed arrivals and routing.
- Service durations and staffing schedules.
- Waiting, abandonment and model validation.

Help improve this course

The course repository is open for collaboration.

- Find the slides, notebook and data. Clone it.
- Report a mistake or an unclear explanation in an issue.
- Suggest a correction or a better example with a pull request.

Open the course repository

`github.com/NaitsabesMue/stochastic-modeling-simulation-ds`



Scan for the repository